

### ■ EXAMPLE 15.1

Suppose that we have a scalar system given by the following equations:

$$\begin{aligned}x_k &= \frac{1}{2}x_{k-1} + \frac{25x_{k-1}}{1+x_{k-1}^2} + 8\cos[1.2(k-1)] + w_k \\y_k &= \frac{1}{20}x_k^2 + v_k\end{aligned}\tag{15.25}$$

where  $\{w_k\}$  and  $\{v_k\}$  are zero-mean Gaussian white noise sequences, both with variances equal to 1. This system has become a benchmark in the nonlinear estimation literature [Kit87, Gor93]. The high degree of nonlinearity in both the process and measurement equations makes this a difficult state estimation problem for a Kalman filter. We take the initial state as  $x_0 = 0.1$ , the initial state estimate as  $\hat{x}_0 = x_0$ , and the initial estimation covariance for the Kalman filter as  $P_0^+ = 2$ . We can simulate the EKF and the particle filter to estimate the state  $x$ . We used a simulation length of 50 time steps, and 100 particles in the particle filter. Figure 15.3 shows the EKF and particle filter estimates of the state. Not only is the EKF estimate poor, but the EKF *thinks* (on the basis of the computed covariance) that the estimate is much better than it really is. The true state is usually farther away from the estimated state than the 95% confidence measure of the EKF (as determined from the covariance  $P$ ). On the other hand, Figure 15.3 shows that the particle filter does a nice job of estimating the state for this example. The RMS estimation errors for the Kalman and particle filters were 16.3 and 2.6, respectively.

Note that it might be possible to modify the Kalman filter to obtain better performance. For example, some of the procedures discussed in Section 5.5 to prevent divergence could improve the Kalman filter performance in this example. Sometimes, changing the coordinate system of the state space equation or measurement equation can improve performance [Aid83]. Nevertheless, this example shows the type of improvement that can be obtained with the use of particle filtering.

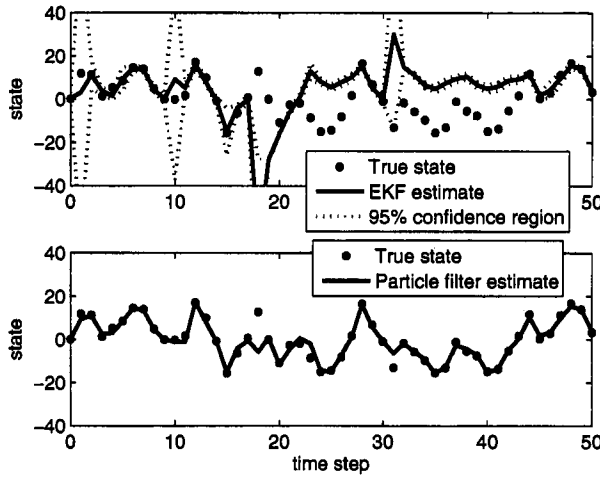
▽▽▽

## 15.3 IMPLEMENTATION ISSUES

In this section, we discuss a few implementation issues that often arise in the application of particle filters. The methods discussed in this section can significantly improve the performance of the particle filter, and in fact can make the difference between success and failure.

### 15.3.1 Sample impoverishment

Sample impoverishment occurs when the region of state space in which the pdf  $p(y_k|x_k)$  has significant values does not overlap with the pdf  $p(x_k|y_{k-1})$ . This means that if all of our *a priori* particles are distributed according to  $p(x_k|y_{k-1})$ , and we then use the computed pdf  $p(y_k|x_k)$  to resample the particles, only a few particles will be resampled to become *a posteriori* particles. This is because only a few of the



**Figure 15.3** Example 15.1 results. Extended Kalman filter and particle filter estimation performance for a highly nonlinear scalar system.

*a priori* particles will be in a region of state space where the computed pdf  $p(y_k|x_k)$  has a significant value. This means that the resampling process will select only a few distinct *a priori* particles to become *a posteriori* particles. Eventually, all of the particles will collapse to the same value.<sup>1</sup> This problem will be exacerbated if the measurements are not consistent with the process model (modeling errors). This can be overcome by a brute-force method of simply increasing the number of particles  $N$ , but this can quickly lead to unreasonable computational demands, and often simply delays the inevitable sample impoverishment. Other more intelligent ways of dealing with this problem can be used [Aru02, Gor93]. In the following subsections we discuss several remedies for sample impoverishment, including roughening, prior editing, regularized particle filtering, Markov chain Monte Carlo resampling, and auxiliary particle filtering.

**15.3.1.1 Roughening** Roughening can be used to prevent sample impoverishment, as shown in [Dou01, Chapter 14], [Gor93]. In this method, random noise is added to each particle after the resampling process. This is similar to adding artificial process noise to the Kalman filter (see Section 5.5). In the roughening approach, the *a posteriori* particles (i.e., the outputs of the resampling step) are modified as follows:

$$\begin{aligned} x_{k,i}^+(m) &= x_{k,i}^+(m) + \Delta x(m) \quad (m = 1, \dots, n) \\ \Delta x(m) &\sim (0, KM(m)N^{-1/n}) \end{aligned} \quad (15.26)$$

$\Delta x(m)$  is a zero-mean random variable (usually Gaussian).  $K$  is a scalar tuning parameter,  $N$  is the number of particles,  $n$  is the dimension of the state space, and  $M$  is a vector containing the maximum difference between the particle elements

<sup>1</sup>This is called the black hole of particle filtering.

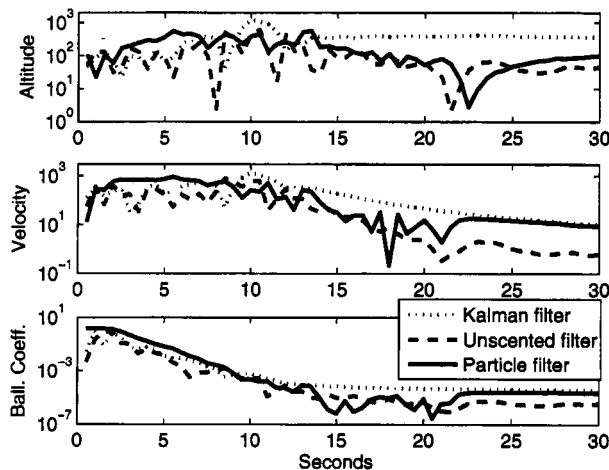
before roughening. The  $m$ th element of the  $M$  vector is given as

$$M(m) = \max_{i,j} |x_{k,i}^+(m) - x_{k,j}^+(m)| \quad (m = 1, \dots, n) \quad (15.27)$$

where  $k$  is the time step, and  $i$  and  $j$  are particle numbers.  $K$  is a tuning parameter that specifies the amount of jitter that is added to each particle. In [Gor93] the value  $K = 0.2$  is used.

### ■ EXAMPLE 15.2

In this example, we consider the same problem as discussed in Example 14.2. That is, we will try to estimate the altitude, velocity, and ballistic coefficient of a body as it falls toward earth. We use the extended Kalman filter, the unscented Kalman filter, and the particle filter to estimate the system state. A straightforward implementation of the particle filter does not work very well in this example. In order to get good results we had to use the roughening procedure of Equation (15.26) with a tuning parameter  $K = 0.2$ . We also had to constrain each particle's third element (ballistic coefficient) to a nonnegative value so that the integration of the time-update equations in the particle filter did not diverge. We used 1000 particles. Figure 15.4 shows typical EKF, UKF, and particle filter estimation error magnitudes for this system. It is seen that the particle filter provides performance on par with the UKF, but at the price of much higher computational effort. The UKF is essentially an “intelligent” particle filter with only seven particles (twice the number of states plus one), whereas the particle filter can be viewed as a “brute-force” filter with 1000 particles. Perhaps some additional modifications could be made to the particle filter to obtain better performance, but the same could be said for the UKF.



**Figure 15.4** Example 15.2 results. Kalman filter, unscented filter, and particle filter estimation-error magnitudes.

**15.3.1.2 Prior editing** If roughening does not prevent sample impoverishment, then prior editing can be tried. This involves rejection of an *a priori* sample if it is in a region of state space with small  $q_i$ . If an *a priori* sample is in a region of small probability, then it can be roughened as many times as necessary, using a procedure like Equation (15.26), until it is in a region of significant probability  $q_i$ . In [Gor93] prior editing is implemented as follows: if the magnitude of  $[y_k - h(x_{k,i}^-)]$  is more than six standard deviations of the measurement noise, then it is highly unlikely to be selected as an *a posteriori* particle. In this case,  $x_{k-1,i}^+$  is roughened and then passed through the system equation again to obtain a new  $x_{k,i}^-$ . This is repeated as many times as necessary until  $x_{k,i}^-$  is in a region of nonnegligible probability.

**15.3.1.3 Regularized particle filtering** Another way of preventing sample impoverishment is through the use of the regularized particle filter (RPF) [Dou01, Chapter 12], [Ris04]. This performs resampling from a continuous approximation of the pdf  $p(y_k|x_{k,i}^-)$  rather than from the discrete pdf samples used thus far. Recall in our resampling step in Equation (15.18) that we used the probability

$$q_i = P[(y_k = y^*)|(x_k = x_{k,i}^-)] \quad (15.28)$$

to determine the likelihood of selecting an *a priori* particle to be an *a posteriori* particle. Instead, we can use the pdf  $p(x_k|y_k)$  to perform resampling. That is, the probability of selecting the particle  $x_{k,i}^-$  to be an *a posteriori* particle is proportional to the pdf  $p(x_k|y_k)$  evaluated at  $x_k = x_{k,i}^-$ . In the RPF, this pdf is approximated as

$$\hat{p}(x_k|y_k) = \sum_{i=1}^N w_{k,i} K_h(x_k - x_{k,i}) \quad (15.29)$$

where  $w_{k,i}$  are the weights that are used in the approximation. Later on, we will see that these weights should be set equal to the  $q_i$  probabilities that were computed in Equation (15.18).  $K_h(\cdot)$  is given as

$$K_h(x) = h^{-n} K(x/h) \quad (15.30)$$

where  $h$  is the positive scalar kernel bandwidth, and  $n$  is the dimension of the state vector.  $K(\cdot)$  is a kernel density that is a symmetric pdf that satisfies

$$\begin{aligned} \int x K(x) dx &= 0 \\ \int \|x\|_2^2 K(x) dx &< \infty \end{aligned} \quad (15.31)$$

The kernel  $K(\cdot)$  and the bandwidth  $h$  are chosen to minimize a measure of the error between the assumed true density  $p(x_k|y_k)$  and the approximate density  $\hat{p}(x_k|y_k)$ :

$$\{K(x), h\} = \operatorname{argmin} \int [\hat{p}(x|y_k) - p(x|y_k)]^2 dx \quad (15.32)$$

In the classic case of equal weights ( $w_{k,i} = 1/N$  for  $i = 1, \dots, N$ ) the optimal kernel is given as

$$K(x) = \begin{cases} \frac{n+2}{2v_n} (1 - \|x\|_2^2) & \text{if } \|x\|_2 < 1 \\ 0 & \text{otherwise} \end{cases} \quad (15.33)$$

where  $v_n$  is the volume of the  $n$ -dimensional unit hypersphere.  $K(x)$  is called the Epanechnikov kernel [Dou01, Chapter 12].

An  $n$ -dimensional unit hypersphere is a volume in  $n$  dimensions in which all points are one unit from the origin [Cox73]. In one dimension, the unit hypersphere is a line with a length of two and a “volume” of two. In two dimensions, the unit hypersphere is a circle with a radius of one and volume  $\pi$ . In three dimensions, the unit hypersphere is a ball with a radius of one and volume  $4\pi/3$ . In  $n$  dimensions, the unit hypersphere has a volume  $v_n = 2\pi v_{n-2}/n$ .

If  $p(x|y_k)$  is Gaussian with an identity covariance matrix then the optimal bandwidth is given as

$$h^* = [8v_n^{-1}(n+4)(2\sqrt{\pi})^n]^{1/(n+4)} N^{-1/(n+4)} \quad (15.34)$$

In order to handle the case of multimodal pdf's,<sup>2</sup> we should use  $h = h^*/2$  [Dou01, Chapter 12],[Sil86]. These choices for the kernel and the bandwidth are optimal only for the case of equal weights and a Gaussian pdf, but they still are often used in other situations to obtain good particle filtering results. Instead of selecting *a priori* particles to become *a posteriori* particles using the probabilities of Equation (15.28), we instead select *a posteriori* particles based on the pdf approximation given in Equation (15.29). This allows more diversity as we perform the update from the *a priori* particles to *a posteriori* particles. In general, we should set the  $w_{k,i}$  weights in Equation (15.29) equal to the  $q_i$  probabilities shown in Equation (15.28).

Since this procedure assumes that the true density  $p(x_k|y_k)$  has a unity covariance matrix, we numerically compute the covariance of the  $x_{k,i}^-$  at each time step. Suppose that this covariance is computed as  $S$  (an  $n \times n$  matrix). Then we compute the matrix square root of  $S$ , denoted as  $A$ , such that  $AA^T = S$  (e.g., we can use Cholesky decomposition for this computation). Then we compute the kernel as

$$K_h(x) = (\det A)^{-1} h^{-n} K(A^{-1}x/h) \quad (15.35)$$

The RPF resampling algorithm can be summarized as follows.

### Regularized particle filter resampling

This resampling strategy replaces Step (3d) in the particle filter algorithm on page 468. We have an  $n$ -state system, the  $N$  *a priori* particles  $x_{k,i}^-$  and the  $N$  corresponding (normalized) *a priori* probabilities  $q_i$ . Generate the *a posteriori* particles  $x_{k,i}^+$  as follows.

1. Compute the ensemble mean  $\mu$  and covariance  $S$  of the *a priori* particles as follows.

$$\begin{aligned} \mu &= \frac{1}{N} \sum_{i=1}^N x_{k,i}^- \\ S &= \frac{1}{N-1} \sum_{i=1}^N (x_{k,i}^- - \mu)(x_{k,i}^- - \mu)^T \end{aligned} \quad (15.36)$$

Some authors use an  $N$  in the denominator of the  $S$  equation, but  $(N-1)$  gives an unbiased estimate (see Problem 3.6).

<sup>2</sup>A multimodal pdf is one with more than one local maxima. See, for example, Figure 15.1.

2. Perform a square root factorization of  $S$  (e.g., a Cholesky factorization) to compute the  $n \times n$  matrix  $A$  such that  $AA^T = S$ .
3. Compute the volume of the  $n$ -dimensional unit sphere as  $v_n = 2\pi v_{n-2}/n$ . The starting values for this recursion are  $v_1 = 2$ ,  $v_2 = \pi$ , and  $v_3 = 4\pi/3$ .
4. Compute the optimal kernel bandwidth  $h$  as follows:

$$h = \frac{1}{2} [8v_n^{-1}(n+4)(2\sqrt{\pi})^n]^{1/(n+4)} N^{-1/(n+4)} \quad (15.37)$$

The bandwidth  $h$  can be considered a tuning parameter for the particle filter.

5. Approximate the pdf  $p(x_k|y_k)$  as follows:

$$\hat{p}(x_k|y_k) = \sum_{i=1}^N q_i K_h(x_k - x_{k,i}) \quad (15.38)$$

where the kernel  $K_h(x)$  is given as

$$K_h(x) = (\det A)^{-1} h^{-n} K(A^{-1}x/h) \quad (15.39)$$

and the Epanechnikov kernel  $K(x)$  is given as

$$K(x) = \begin{cases} \frac{n+2}{2v_n} (1 - \|x\|_2^2) & \text{if } \|x\|_2 < 1 \\ 0 & \text{otherwise} \end{cases} \quad (15.40)$$

Note that other kernels can also be used in the pdf approximation (see Problem 15.14). Equation (15.38) must be implemented digitally, so the user must choose a certain number of digital values at which to evaluate Equation (15.38). As with the number of particles  $N$ , the number of digital values is a trade-off between computational resources and estimation accuracy.

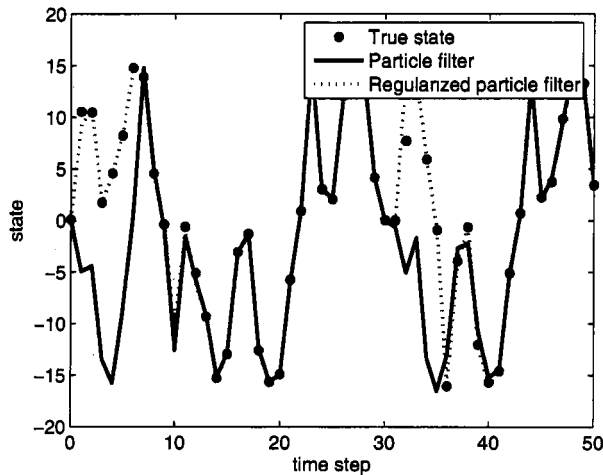
6. Now that we have  $\hat{p}(x_k|y_k)$  from the previous step, we generate the *a posteriori* particles  $x_{k,i}^+$  by probabilistically selecting points from the pdf approximation  $\hat{p}(x_k|y_k)$ .

### ■ EXAMPLE 15.3

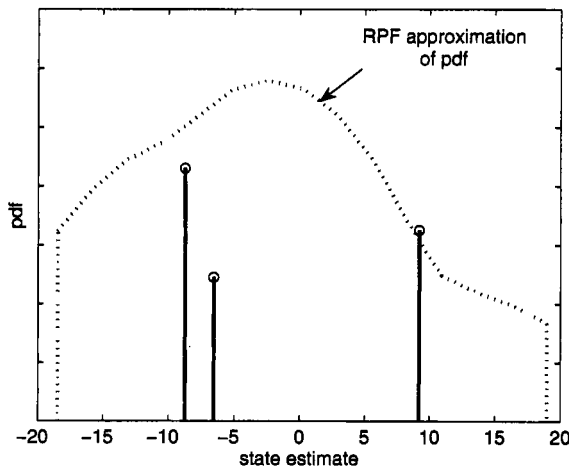
Consider the same system as in Example 15.1, except use a process-noise covariance of 0.001 and only three particles ( $N = 3$ ). In this case, the particles in the standard particle filter can quickly degenerate into a single point, but the use of an RPF can prevent this degeneration, increase diversity among the particles, and provide a better state estimate. Twenty Monte Carlo runs of this system result in average RMS errors of 4.6 for the standard particle filter and 3.0 for the RPF. Figure 15.5 shows the improvement that is possible with the use of an RPF.

Figure 15.6 shows the difference between the resampling step of the standard particle filter and the RPF. The standard particle filter has an *a priori* pdf approximation that consists of the sum of impulse functions. Therefore, the *a posteriori* particles are all set equal to one of the *a priori* particles.

However, the RPF has a pdf approximation that is a continuous function of the state estimate. Therefore, the *a posteriori* particles can be equal to any value on the horizontal axis. Of course, when we implement the RPF we have to discretize the horizontal axis in order to choose the *a posteriori* particles, but we can use as fine a discretization as our computational resources will allow.



**Figure 15.5** Particle filter estimation performance for the highly nonlinear scalar system of Example 15.3. This shows the improvement that is possible with the use of an RPF.



**Figure 15.6** This shows the discrete pdf approximation of the standard particle filter (with three particles), and the continuous pdf approximation of the RPF. This plot is a snapshot of the pdf approximations at one time instant.

**15.3.1.4 Markov chain Monte Carlo resampling** Another approach for preventing sample impoverishment is the Markov chain Monte Carlo (MCMC) move step [Gil01, Ris04]. This approach moves the *a priori* particle  $x_{k,i}^-$  to a new randomly generated state  $\tilde{x}_{k,i}^-$  if a uniformly distributed random number is less than an acceptance probability. The acceptance probability is computed as the probability that the *a priori* sample is consistent with the measurement, relative to the probability that the resampled state is consistent with the measurement. The Metropolis–Hastings acceptance probability [Rob99] is given as

$$\alpha = \min \left[ 1, \frac{p(y_k | \tilde{x}_{k,i}^-) p(\tilde{x}_{k,i}^- | x_{k-1,i}^+)}{p(y_k | x_{k,i}^-) p(x_{k,i}^- | x_{k-1,i}^+)} \right] \quad (15.41)$$

The first fraction in the above equation is the ratio of the measurement probability conditioned on the new particle to the measurement probability conditioned on the old particle. The second fraction is the ratio of the probability of the new particle to the probability of the old particle, both conditioned on the particle at the previous time. The acceptance probability is the product of these two fractions, which increases as the probability of the new particle increases. The old *a priori* particle  $x_{k,i}^-$  is therefore changed to a new particle  $\tilde{x}_{k,i}^-$  if the old particle has a low probability of being selected with the resampling step. This helps to maintain diversity in the particles that come out of the resampling step.

**15.3.1.5 Auxiliary particle filtering** Another approach to evening out the probability of the *a priori* particles (and thus increasing diversity in the *a posteriori* particles) is called the auxiliary particle filter [Pit99, Ris04]. This approach was developed by augmenting each *a priori* particle by one element (an auxiliary variable). This increases the dimension of the problem and thus adds a degree of freedom to the choice of the resampling weights in Equation (15.19), which allows the resampling weights to be more evenly distributed. Recall from Section 15.2 that the resampling step of the standard particle filter is performed by selecting particles based on their probabilities. These probabilities are given by

$$q_i = P[(y_k = y^*) | (x_k = x_{k,i}^-)] \quad (15.42)$$

where  $y^*$  is the actual measurement at time  $k$ . The problem with this is that outliers in the batch of *a priori* particles are ignored due to their low probabilities, and the particles can therefore collapse into a single point. Auxiliary particle filtering addresses this issue by changing the resampling probability to the following:

$$q_i = \frac{P[(y_k = y^*) | (x_k = x_{k,i}^-)]}{P[(y_k = y^*) | \mu_{k,i}]} \quad (15.43)$$

where  $\mu_{k,i}$  is some statistical characterization of  $x_k$  based on  $x_{k,i}^-$ . For example, we could use  $\mu_{k,i} = E(x_k | x_{k,i}^-)$ , or  $\mu_{k,i} = \text{pdf}(x_k | x_{k,i}^-)$ . So compared to the standard particle filter, the resampling probability of the auxiliary particle filter is smaller by a factor of  $P[(y_k = y^*) | \mu_{k,i}]$ . If the actual measurement is highly likely given  $\mu_{k,i}$ , then the actual measurement is highly likely given  $x_{k,i}^-$ . The auxiliary particle filter will then tend to decrease  $q_i$  relative to the standard particle filter. Likewise, the auxiliary particle filter will tend to increase  $q_i$  for highly unlikely particles. This tends to promote diversity in the population of particles.



Another easy way to smooth out the  $q_i$  probabilities is to use something like the following formula.

$$\tilde{q}_i = \frac{(\alpha - 1)q_i + \bar{q}}{\alpha} \quad (15.44)$$

where  $\bar{q}$  is the sample mean of all of the  $q_i$  probabilities. The parameter  $\alpha \in [1, \infty]$  controls how much regularization occurs. If  $\alpha \rightarrow \infty$  then the regularized probabilities  $\tilde{q}_i$  are equal to the standard probabilities  $q_i$ . If  $\alpha = 1$  then all of the regularized probabilities  $\tilde{q}_i$  are equal.

If the dynamics of the state-space system are linear, then there should not be any reason to use auxiliary particle filtering. The existence of outliers in the particles results from nonlinearities. This implies that the use of auxiliary particle filtering is more appropriate when the system nonlinearities are severe. In fact, if the nonlinearities are mild or nonexistent, then the use of auxiliary particle filtering could corrupt the probabilities  $q_i$  in an inappropriate way and degrade performance relative to the standard particle filter.

#### ■ EXAMPLE 15.4

Consider the same system as in Examples 14.2 and 15.2. That is, we will try to estimate the altitude  $x_1$ , velocity  $x_2$ , and constant ballistic coefficient  $x_3$  of a body as it falls toward earth. The equations for this system are given in Example 14.2. We use fourth-order Runge–Kutta integration with a step size of 0.5 sec to simulate the system for 30 seconds. We estimate the system states with the standard particle filter and the auxiliary particle filter. As mentioned in Example 15.2, a straightforward implementation of the particle filter does not work very well in this example. In Example 15.2, we used the roughening procedure of Equation (15.26). In this example, we use the auxiliary particle filter of Equation (15.44) with 200 particles. In the standard particle filter, the particles quickly collapse to a single point in state space. In the auxiliary particle filter with  $\alpha = 1.1$  (obtained by manual tuning) the diversity of the particles is preserved. Averaged over 10 simulations, the use of the auxiliary particle filter improves altitude estimation by 73%, and improves velocity estimation by 55%. However, the auxiliary particle filter makes the estimate of the ballistic coefficient worse. This may be because the ballistic coefficient is not involved in any nonlinear dynamics in either the system equation or the measurement equation.

▽▽▽

#### 15.3.2 Particle filtering combined with other filters

One approach that has been proposed for improving particle filtering is to combine it with another filter such as the EKF or the UKF [Wan01, Ris04]. In this approach, each particle is updated at the measurement time using the EKF or the UKF, and then resampling is performed using the measurement. This is like running a bank of  $N$  Kalman filters (one for each particle) and then adding a resampling step after each measurement. After  $x_{k,i}^-$  is obtained as shown in Equation (15.17), it can be refined using the EKF or UKF measurement-update equations. For example, if we want to combine the particle filter with the EKF, then after the measurement is

obtained at time  $k$ ,  $x_{k,i}^-$  is updated to  $x_{k,i}^+$  according to the EKF equations shown in Section 13.2:

$$\begin{aligned} P_{k,i}^- &= F_{k-1,i} P_{k-1,i}^+ F_{k-1,i}^T + Q_{k-1} \\ K_{k,i} &= P_{k,i}^- H_{k,i}^T (H_{k,i} P_{k,i}^- H_{k,i}^T + R_k)^{-1} \\ x_{k,i}^+ &= x_{k,i}^- + K_{k,i} [y_k - h(x_{k,i}^-)] \\ P_{k,i}^+ &= (I - K_{k,i} H_{k,i}) P_{k,i}^- \end{aligned} \quad (15.45)$$

$K_{k,i}$  is the Kalman gain for the  $i$ th particle, and  $P_{k,i}^-$  is the *a priori* estimation-error covariance for the  $i$ th particle. The partial derivative matrices  $F$  and  $H$  are defined as

$$\begin{aligned} F_{k-1,i} &= \left. \frac{\partial f}{\partial x} \right|_{x=x_{k-1,i}^+} \\ H_{k,i} &= \left. \frac{\partial h}{\partial x} \right|_{x=x_{k,i}^-} \end{aligned} \quad (15.46)$$

Next, resampling is performed as discussed in Section 15.2 to modify the  $x_{k,i}^+$  particles (and their associated covariances  $P_{k,i}^+$ ). This is another way to prevent sample impoverishment because the *a priori* particles  $x_{k,i}^-$  are updated on the basis of the measurement at time  $k$  before they are resampled. The measurement updates of the particles could be performed with any type of filter – an EKF, a UKF, an  $H_\infty$  filter, another particle filter, and so on. The extended Kalman particle filter can be summarized as follows.

### The extended Kalman particle filter

1. The system and measurement equations are given as follows:

$$\begin{aligned} x_{k+1} &= f_k(x_k, w_k) \\ y_k &= h_k(x_k, v_k) \end{aligned} \quad (15.47)$$

where  $\{w_k\}$  and  $\{v_k\}$  are independent white noise processes with known pdf's.

2. Assuming that the pdf of the initial state  $p(x_0)$  is known, randomly generate  $N$  initial particles on the basis of the pdf  $p(x_0)$ . These particles are denoted  $x_{0,i}^+$  and their covariances are denoted  $P_{0,i}^+ = P_0^+$  ( $i = 1, \dots, N$ ). The parameter  $N$  is chosen by the user as a trade-off between computational effort and estimation accuracy.
3. For  $k = 1, 2, \dots$ , do the following.

- (a) Perform the time propagation step to obtain *a priori* particles  $x_{k,i}^-$  and covariances  $P_{k,i}^-$  using the known process equation and the known pdf of the process noise:

$$\begin{aligned} x_{k,i}^- &= f_{k-1}(x_{k-1,i}^+, w_{k-1}^i) \\ P_{k,i}^- &= F_{k-1,i} P_{k-1,i}^+ F_{k-1,i}^T + Q_{k-1} \\ F_{k-1,i} &= \left. \frac{\partial f}{\partial x} \right|_{x=x_{k-1,i}^+} \end{aligned} \quad (15.48)$$

where each  $w_{k-1}^i$  noise vector is randomly generated on the basis of the known pdf of  $w_{k-1}$ .

- (b) Update the *a priori* particles and covariances to obtain *a posteriori* particles and covariances:

$$\begin{aligned} H_{k,i} &= \left. \frac{\partial h}{\partial x} \right|_{x=x_{k,i}^-} \\ K_{k,i} &= P_{k,i}^- H_{k,i}^T (H_{k,i} P_{k,i}^- H_{k,i}^T + R_k)^{-1} \\ x_{k,i}^+ &= x_{k,i}^- + K_{k,i} [y_k - h(x_{k,i}^-)] \\ P_{k,i}^+ &= (I - K_{k,i} H_{k,i}) P_{k,i}^- \end{aligned} \quad (15.49)$$

- (c) Compute the relative likelihood  $q_i$  of each particle  $x_{k,i}^+$  conditioned on the measurement  $y_k$ . This is done by evaluating the pdf  $p(y_k|x_{k,i}^+)$  on the basis of the nonlinear measurement equation and the pdf of the measurement noise.
- (d) Scale the relative likelihoods obtained in the previous step as follows:

$$q_i = \frac{q_i}{\sum_{j=1}^N q_j} \quad (15.50)$$

Now the sum of all the likelihoods is equal to one.

- (e) Refine the set of *a posteriori* particles  $x_{k,i}^+$  and covariances  $P_{k,i}^+$  on the basis of the relative likelihoods  $q_i$ . This is the resampling step.
- (f) Now we have a set of *a posteriori* particles  $x_{k,i}^+$  and covariances  $P_{k,i}^+$ . We can compute any desired statistical measure of this set of particles. We typically are most interested in computing the mean and the covariance.

### ■ EXAMPLE 15.5

Consider the same system as in Example 15.4. That is, we will try to estimate the altitude  $x_1$ , velocity  $x_2$ , and constant ballistic coefficient  $x_3$  of a body as it falls toward earth. The equations for this system are given in Example 14.2. We use fourth-order Runge–Kutta integration with a step size of 0.5 sec to simulate the system for 30 s. We use the standard particle filter and the EKF particle filter to estimate the states. The EKF particle filter updates the *a priori* particles at each time based on the measurement, and then the resampling step is performed as usual. In this example, we use 200 particles for the estimator. As mentioned in Example 15.4, in the standard particle filter the particles quickly collapse to a single trajectory. In Example 15.2, we used roughening to improve the particle filter. In Example 15.3, we used the regularized particle filter to improve performance. In Example 15.4, we used the auxiliary particle filter to improve performance. Here we use an EKF particle filter to improve performance. Averaged over 10 simulations, the use of the EKF particle filter improves altitude estimation accuracy by an astounding 99.6%, almost three orders of magnitude. The velocity estimation is only marginally improved, and the ballistic coefficient estimation is marginally degraded.